

Proyecto Final – Estación de Monitoreo Ambiental IoT (ESP32)

Reporte de trabajo · Curso de Microcontroladores

Autor: Xólotl Gael Chávez Zamora · **Placa:** ESP32 DevKit-C v4 · **Simulador:** Wokwi · **Entorno:** Arduino (WiFi.h + WebServer.h) · **Fecha:** 13/09/2026

1. Repositorio web (alojamiento en GitHub)

Todo el material — código de los **proyectos guiados** y del **proyecto final**, el **video**, los **diagramas** y este reporte — está alojado en un repositorio **público** (no requiere permisos de acceso):

Repositorio: <https://github.com/xgael/evidencias-microcontroladores>

Recurso	Enlace
 Video de demostración (≥ 2 min)	/proyecto-final-iot-esp32/video/...Demo.mp4
 Código del proyecto final (sketch + circuito)	/proyecto-final-iot-esp32/codigo
 Diagramas (flujo, bloques, circuito)	/proyecto-final-iot-esp32/diagramas
 Proyectos guiados (Blink, Semáforo, Luz nocturna)	/proyectos-guiados

2. Descripción del proyecto

El proyecto es una **estación de monitoreo ambiental IoT** construida sobre un microcontrolador **ESP32**. El sistema mide temperatura y luz, evalúa la temperatura contra un **umbral configurable** mediante un **semáforo térmico** de tres LEDs, dispara una **alarma sonora** (buzzer) al superar el umbral y expone toda la información y el control a través de un **servidor web embebido** accesible desde cualquier navegador de la red.

Objetivo

Integrar en un solo microcontrolador la lectura de sensores analógicos, la lógica de control con actuadores y un servidor web que permita monitoreo y control remotos, demostrando los conceptos de IoT vistos en el curso.

Componentes

Componente	Pin ESP32	Función
Sensor de temperatura NTC 10 kΩ	GPIO34 (ADC)	Entrada analógica — temperatura
Fotorresistencia (LDR)	GPIO35 (ADC)	Entrada analógica — nivel de luz

Botón "Silenciar"	GPIO19 (pull-up)	Entrada digital — silenciar alarma
LED verde / amarillo / rojo	GPIO25 / 26 / 27	Semáforo térmico (220 Ω c/u)
LED azul (actuador remoto)	GPIO14	Salida controlada por web (220 Ω)
Buzzer	GPIO18	Alarma sonora (tono 1 kHz)

Servidor web embebido

El ESP32 se conecta por WiFi y levanta un servidor HTTP en el **puerto 80** (librería `WebServer.h`) con las siguientes rutas:

Ruta	Método	Función
/	GET	Dashboard HTML responsivo
/data	GET	Telemetría en JSON (consultada cada 1 s por el navegador)
/led?state=on off	GET	Enciende/apaga el actuador remoto (LED azul)
/umbral?c=NN	GET	Cambia el umbral de alarma
/silenciar	GET	Silencia la alarma activa

3. Diagrama de bloques (funcionamiento)

Explica la funcionalidad del sistema: el flujo de señales de los **sensores** hacia el **ESP32** (ADC → conversión física → lógica de control), y de éste hacia los **actuadores**, con acceso remoto bidireccional vía HTTP desde el navegador. **Incluye anotaciones**

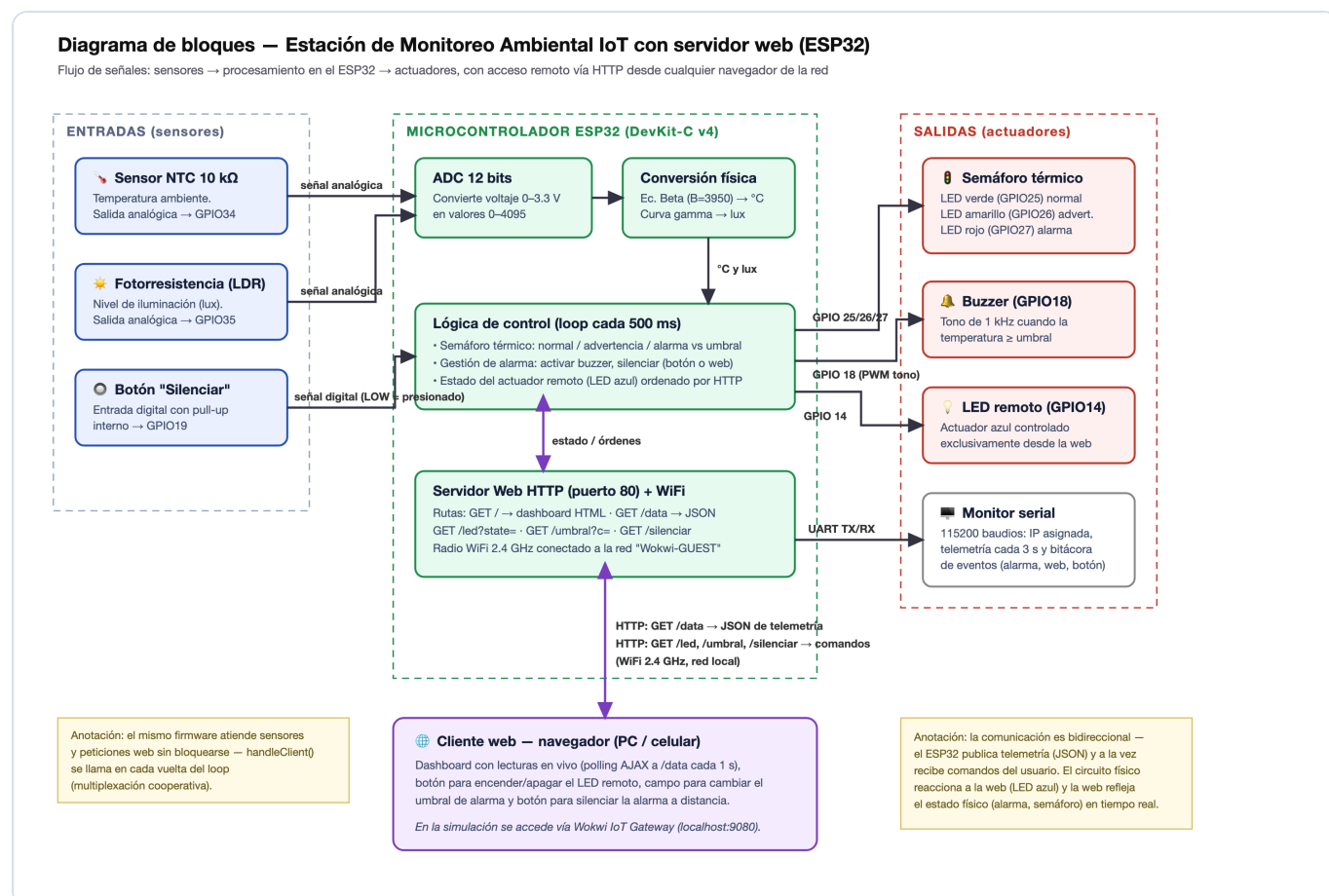


Figura 1. Diagrama de bloques anotado — entradas, procesamiento en el ESP32, salidas y cliente web.

4. Diagrama de flujo del programa

Detalla el ciclo del firmware: inicialización, conexión WiFi, arranque del servidor y el **loop** principal (atención de peticiones HTTP, lectura de sensores cada 500 ms y actualización del semáforo/alarma).

Incluye anotaciones

Diagrama de flujo — Estación de Monitoreo Ambiental IoT (ESP32)

Proyecto final · sketch.ino · Las notas al margen explican cada etapa del programa

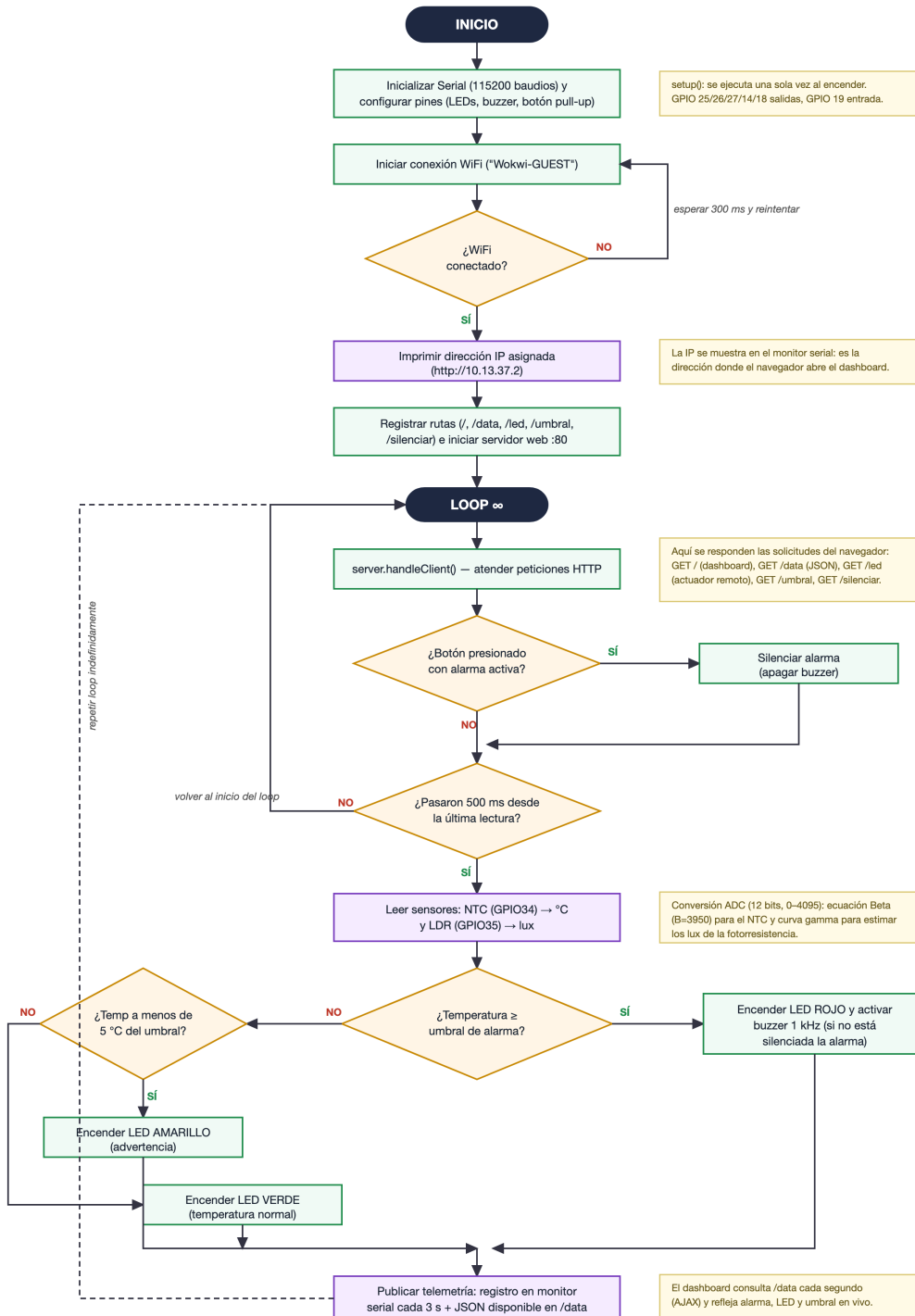


Figura 2. Diagrama de flujo anotado del programa (solo la actividad desarrollada).

5. Diagrama de circuito

Se incluye el diagrama de conexiones anotado (pin por pin) y una captura del circuito en el simulador **Wokwi**. El circuito es reproducible importando el archivo `diagram.json` del repositorio en wokwi.com/projects/new/esp32.

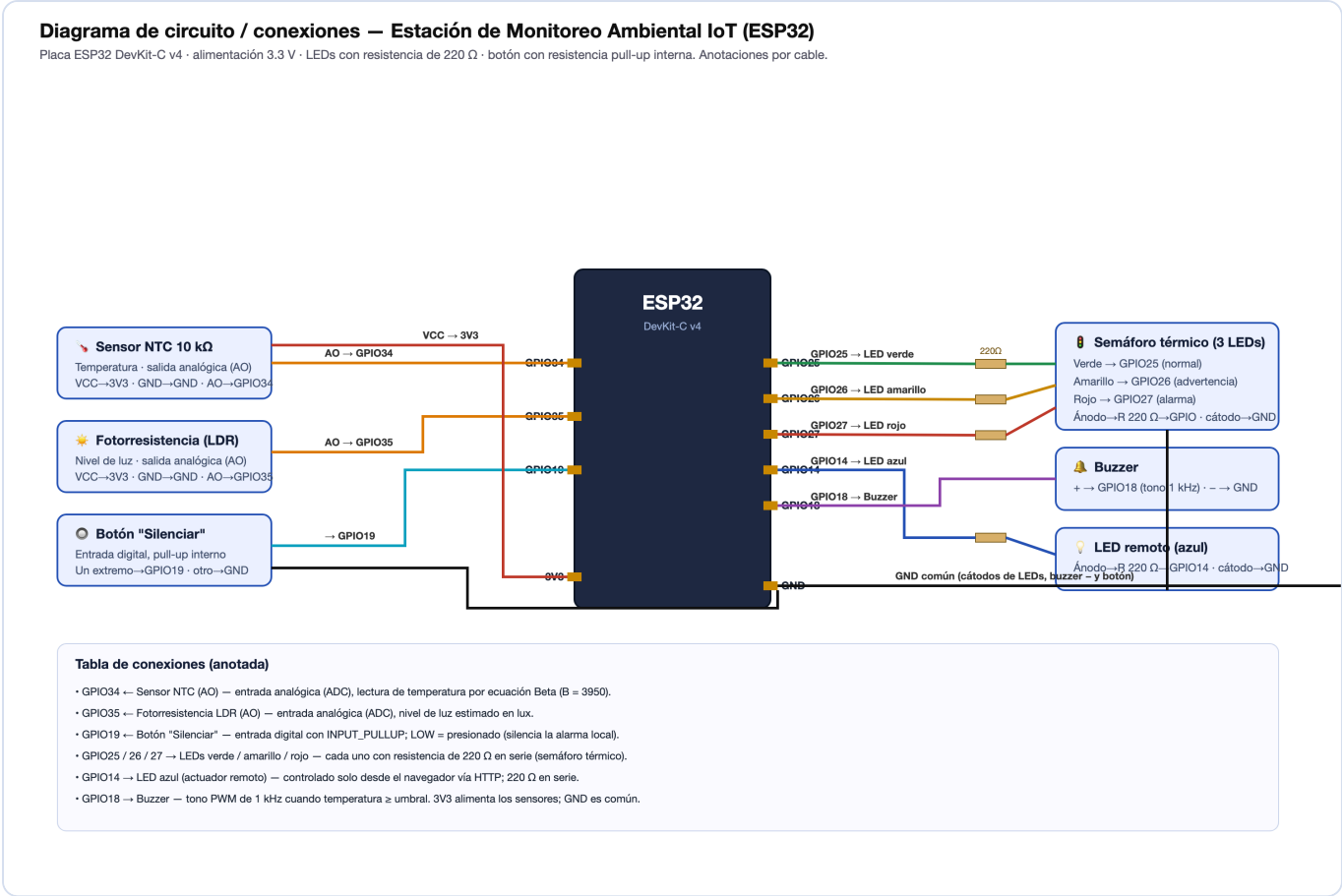


Figura 3. Diagrama de circuito / conexiones anotado (pines, resistencias de 220 Ω, GND común).

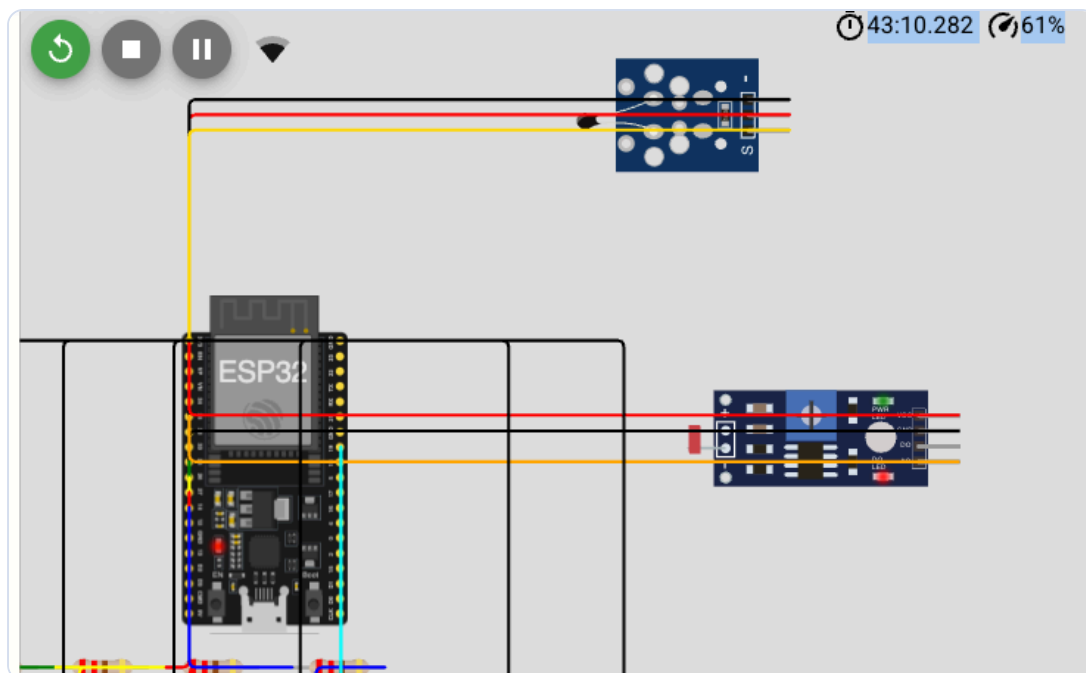


Figura 4. Captura del circuito montado y en ejecución en Wokwi (ESP32 + sensores + semáforo + buzzer).

Enlace al circuito: definición del circuito en [codigo/diagram.json](#) — se abre en Wokwi importándolo en un proyecto ESP32 nuevo.

6. Video de demostración

El video (2:05 min) muestra: la simulación en Wokwi con el circuito y el monitor serial (el ESP32 conectándose a WiFi y levantando el servidor en el puerto 80 + telemetría), el **dashboard servido por el ESP32** con datos en vivo, el semáforo térmico reaccionando (NORMAL → ADVERTENCIA → ALARMA), el silenciado remoto de la alarma, el control remoto del LED y el cambio de umbral, y finalmente los diagramas de bloques y de flujo.

 **Video:** [Proyecto Final IoT ESP32 Demo.mp4](#)

7. Proyectos guiados

Se incluyen los proyectos guiados del curso, cada uno con su `sketch.ino` y su `diagram.json` para Wokwi:

- **01 · Blink** — parpadeo de un LED (E/S digital, temporización).
- **02 · Semáforo** — secuencia verde/amarillo/rojo con temporización.
- **03 · Luz nocturna automática** — lectura analógica de una LDR y decisión de encendido.

Guiados en GitHub: [/proyectos-guiados](#)